

# Bash Scripts for Hands-on Practice

## Script 1: System Info Reporter

### Requirements:

- Accept one optional flag: -v for verbose mode (use getopt)
- Display: hostname, current user, OS version, uptime, current date/time
- Use at least 5 built-in variables (\$HOSTNAME, \$USER, \$HOME, \$PWD, \$\$)
- Store each piece of info in a named variable before printing
- In verbose mode, also display: CPU model, total RAM, disk usage
- Output all info to both terminal and a log file (system\_info.log)
- Append to log with timestamp, do not overwrite previous runs
- If log file does not exist, create it automatically
- Exit with code 0 on success, 1 on failure

## Script 2: User Account Manager

### Requirements:

- Accept a command-line argument: create / delete / list (use \$1)
- If no argument provided, print usage instructions and exit with code 1
- create: prompt for username, check if user already exists (-eq 0 check on \$?)
  - If exists, print warning and exit
  - If not, create user and set password, confirm with success message
- delete: prompt for username, confirm deletion (y/n input validation)
  - Use case statement for y/Y/n/N handling
- list: display all users with UID >= 1000 from /etc/passwd
  - Use while loop with read to parse /etc/passwd line by line
  - Extract and display only username and home directory fields
- Log every action with timestamp to /var/log/user\_manager.log
- Use a function for each operation (create\_user, delete\_user, list\_users)
- Use trap to catch Ctrl+C and print "Operation cancelled" before exiting

## Script 3: Disk and Mount Monitor

### Requirements:

- Define an array of mount points to monitor (e.g. / /home /boot)
- Use a for loop to iterate over each mount point
- For each mount point:
  - Check if it exists using file test operator (-d)
  - If not mounted, print warning with the mount point name
  - If mounted, display: total size, used, available, usage percentage
- Set a threshold variable (e.g. THRESHOLD=80)
- If any mount point usage exceeds threshold, print an alert message
- Use a function called check\_disk that takes a mount point as argument
- Redirect all warnings and alerts to stderr (>&2)
- Redirect normal output to a report file: disk\_report.log
- At the end, print total number of mount points checked
- Accept -t flag to override default threshold from command line

#### Script 4: Service Health Checker with Auto-Restart

##### Requirements:

- Define an array of critical services to monitor (e.g. ssh cron networking)
- Use a while true loop that runs every 30 seconds (use sleep)
- In each iteration, use a for loop to check each service
- Check service status using systemctl is-active
- Use if-elif-else to handle three states: active / inactive / unknown
- If inactive: attempt to restart, log the attempt
  - Check if restart succeeded (\$?)
  - If restart failed, log as CRITICAL
  - If restart succeeded, log as RECOVERED
- Use a function called log\_event that accepts severity and message
  - log\_event formats: [TIMESTAMP] [SEVERITY] MESSAGE
  - Append to /var/log/service\_monitor.log
- Use trap SIGTERM and SIGINT to gracefully stop the loop and log shutdown
- Count total restarts attempted and display on exit
- Use set -u to catch any undefined variable bugs

#### Script 5: Automated Backup System

##### Requirements:

- Accept source directory and backup destination as positional parameters (\$1 \$2)
- Validate both arguments exist: check with -d for directories
- If either is missing or invalid, print error to stderr and exit 1
- Use date command with command substitution to generate timestamped backup name
- Use rsync to perform incremental backup (preserve permissions, show progress)
- Capture rsync exit code and handle success / failure separately
- After backup, calculate and display: number of files backed up, total size
- Use a here document to write a backup summary report file
- Maintain only the last 7 backups: use ls and array to find and delete old ones
- Use a function called cleanup\_old\_backups that takes destination as argument
- Inside cleanup function, use a while loop with counter
- Schedule itself: output the correct crontab line to run daily at 2am
- Use set -e so any unexpected error stops the script immediately
- Use trap to remove temporary files on exit (even on error)